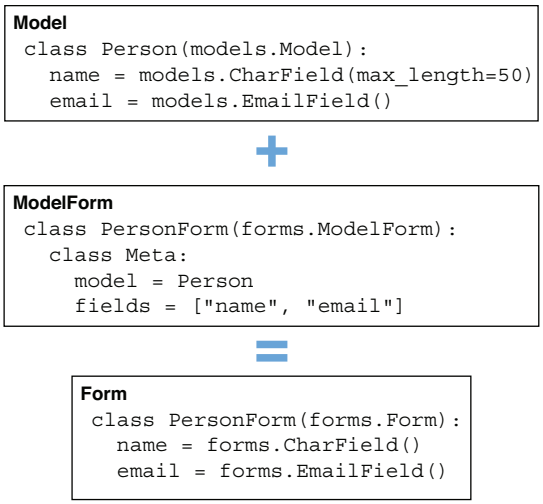


Figure 7.4 A `ModelForm` generates a `Form` equivalent for you based on a model class

You can customize things further, though, if you wish. When you use a `Form` class to build a form, you can modify each field as part of their declaration. You can change the field's label, help text, or which widget renders the result. A `ModelForm` declares the fields for you, so you don't have the opportunity to change these attributes at declaration time. Instead, you can override the `.__init__()` method.

Both the `Form` and `ModelForm` classes have an attribute called `fields`, which is a dictionary containing the fields on the form. By overriding `.__init__()`, you can modify any field after it has been constructed through the use of this dictionary. This means you can change the label, the help text, or the widget that renders the tag.

Open RiffMates/content/forms., and add the `SeekingAdForm` class found in the following listing. This class creates a form based on the `SeekingAd` model and modifies some of the labels and help text of the corresponding fields.

Listing 7.7 Django `ModelForm` for the `SeekingAd` ORM model

```
# RiffMates/content/forms.py
...
from content.models import SeekingAd
...

class SeekingAdForm(forms.ModelForm):
    class Meta:
        model = SeekingAd
        fields = ["seeking", "musician", "band",
            "content", ]

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.fields["seeking"].label = "I am seeking a"
```

Associate SeekingAdForm with the SeekingAd ORM model.

A list of the fields from SeekingAd that should appear in the form

Modify the label for the seeking field.